



Grado en Matemáticas

RESOLUTOR DE LABERINTOS EN HASKELL

-18-12-2024-

Autor: Sergio de María Saiz

Manual de Usuario

Nuestra entrega consiste en un resolutor de laberintos. Para comenzar a usar el programa, una vez cargado en ghci se usa la función *play* que guía al usuario a lo largo de la ejecución.

El programa tiene en cuenta que se esta trabajando con windows en lo referido a las rutas, comenzamos preguntando por el nombre o la ruta a un laberinto en formato ".txt" donde las casillas se representan, OJO no hay que indicar el tipo de archivo por defecto le agregamos ".txt".^{al} final:

$$casillas = \begin{cases} A & \text{representa el inicio del laberinto,} \\ B & \text{representa el inicio del laberinto,} \\ \# & \text{representa las paredes del laberinto,} \\ \text{Espacios en blanco} & \text{representa los posibles del laberinto,} \end{cases}$$

Así un ejemplo de laberinto es (adjuntado en la carpeta de casos de prueba):

```
#####
#    ##A   ###
#          #  #
# # ##### # #
#  # ## #  ##
#    ###   #
#### #     #
#  #     ## #
#   # ## #  #
#  #     ## #
#          B#
#####
```

maze1.txt en los casos de prueba

Una vez cargado el laberinto (que guarda en el tipo de dato `maze`), el programa plantea tres métodos de búsqueda mediante un menú:

1. anchura BFS
2. profundidad DFS
3. Greedy A*

Cabe decir que puede que las soluciones pueden ser distintas en función del método usado para resolverlas pues la principal diferencia es el tipo de estructura de datos que se utiliza para almacenar las celdas a explorar.

1. BFS: utiliza una cola para almacenar los nodos a explorar, es decir se exploran antes todos los nodos vecinos del nodo inicial antes de avanzar.
2. DFS: utiliza una pila para almacenar los nodos a explorar, en este caso se recorre todo un camino completo hasta ver que no tiene solución antes de optar por otro.
3. Greedy A*: el más sofisticado de los tres, usa una cola de prioridad, explorando antes las celdas vecinas más cercanas a la meta.

En caso de que no se introduzca ninguna de las opciones (numero enteros entre 1 y 3) no se resuelve el laberinto y devolvemos el laberinto tal cual se recoge del fichero.

Una vez se introduce el método de resolución el programa intenta encontrar una solución, notifica si la encuentra o no, y en caso afirmativo la muestra por pantalla preguntando si deseas guardarla en dos formatos:

1. txt que guarda el texto de la solución, con el mismo formato que vimos para la entrada pero usando * para marcar el camino solución, en un fichero con el nombre especificado por el usuario
2. html con un poco de css para que sean mas vistosos lo que genera un archivo con el nombre especificado por el usuario en la carpeta donde se encuentra la practica. Además al guardar en html tienes la opción de colorear las celdas por las cuales el resolutor paso intentando resolver el laberinto
3. no guardar la solución

Con esto concluye la ejecución de play y podríamos repetir todo el bucle del programa comenzando en play

Recomendación

Con el fin de mantener un orden en la carpeta donde se encuentra la practica recomendamos almacenar los laberintos en una carpeta, por ejemplo `lab`, y al guardar las soluciones indicar en el nombre la dirección de una carpeta (previamente creada), ejemplo de entrada de nombre para guardar: `sol/maze1s`.

Además los primeros ejemplos son mas sencillos para ver las diferencias entre los algoritmos mas fácilmente, los últimos 3 ejemplos son mas grandes y por ello recomendamos visualizarlos siempre en html, (que se abre mediante cualquier navegador web).

Detalles técnicos del algoritmo

Señalamos que a la hora de definir los tipos de datos `Node` y `Maze` hemos usado una notación distinta a la usada en clase. Esta nos permite acceder directamente a las partes del dato sin necesidad de definir funciones auxiliares.

Ejemplo de ejecución:

```
PS C:\Users\alfon\OneDrive - Universidad Complutense de Madrid (UCM)\4º\1º cuatri\Programacion declarativa\entrega\presentar> ghci .\solver.hs
GHCi, version 9.4.8: https://www.haskell.org/ghc/  :? for help
[1 of 2] Compiling Main                ( solver.hs, interpreted )
Ok, one module loaded.
ghci> play
Dame el nombre o ruta con / del fichero del laberinto (Sin extension ej: lab/maze1):
```

Ejecutamos el programa

```
Dame el nombre o ruta con / del fichero del laberinto (Sin extension ej: lab/maze1): lab/maze1

#####
#   ##A ###
#       #
#   #### #
#   ## #
#   ## #
#####
#   ## #
#   ## #
#   ## #
#       B#
#####

¿Qué tipo de búsqueda quieres usar?
1. Búsqueda en anchura BFS (QueueFrontier)
2. Búsqueda en profundidad DFS (StackFrontier)
3. Búsqueda Greedy A* (A*Frontier)
Elige una opción (1, 2 o 3):
```

introducimos el laberinto desde la carpeta lab

```
¡Tengo una solución!

#####
#   ##A ###
#*****#
#**# #### #
#**# ## #
#***## #
#####
#   ## #
#   ## #
#   ## #
#   ***# #
#       ***B#
#####

¿Quieres guardar la solución?
1. Sí, en formato texto
2. Sí, en formato HTML
3. No deseo guardar la solución
Escribe tu opción (1, 2 o 3): 2
Escriba el nombre del fichero para guardar la solución (sin extensión): sol/maze1s
¿Quieres ver los caminos explorados?
1. Sí
2. No
Escribe tu opción (1 o 2): 1
Laberinto con solución guardado en HTML: sol/maze1s.html
```

fin de la ejecución guardamos solución en carpeta sol